

FastPlayer PRO

Desktop Media Player Architecture & Engineering (Lab 1)

คู่มือสถาปัตยกรรมระบบเครื่องเล่นเสียงและวิดีโอเดสก์ท็อป การเรนเดอร์ไฮคอนเวกเตอร์บริสุทธิ์ และการจัดเก็บสถานะ Persistent JSON

01 / VECTOR ICON ENGINE

เรนเดอร์ด้วย QPainter 100%

วาดสัญลักษณ์ Play, Pause, Next, Volume ด้วยสมการเวกเตอร์คมชัดทุก

02 / ZERO DATA LOSS

Persistent JSON Storage

จัดเก็บและกู้คืนสถานะ Playlist, Volume, Liked Songs และคิวเพลงข้ามการเปิด

03 / UNDER BAR DOCK

Interactive Scrubbing

แถบควบคุมมัลติมีเดียด้านล่าง ค้นหาตำแหน่งเพลง Seek Scrubbing ลี

04 / PRODUCTION ENGINE

PyQt6 & QtMultimedia

เชื่อมต่อ Audio Output ระดับเนทีฟ ข้ามแพลตฟอร์ม พร้อมระบบจัดการข้อผิดพลาด

LEAD ARCHITECT & AUTHOR

ผู้จัดทำ: นายเกียรติภูมิ หาศรีนาถ

รหัสนักศึกษา: 64332110242-2 • Project Creator & System Architect

บทที่ 1: วัตถุประสงค์และข้อกำหนดเชิงวิศวกรรม (Engineering Specifications)

1.1 ปัญหาและวัตถุประสงค์เชิงระบบ (Problem Statement & Objectives)

เครื่องเล่นมัลติมีเดียเดสก์ท็อปส่วนใหญ่มักประสบปัญหาการแตกของไอคอนเมื่อเปิดบนหน้าจอ Hi-DPI และข้อมูลเพลย์ลิสต์สูญหายเมื่อปิดโปรแกรม FastPlayer PRO จึงถูกออกแบบเพื่อแก้ไขปัญหานี้ โดยใช้ Pure Vector Painting ร่วมกับ Persistent JSON Storage เพื่อการันตี Zero Data Loss

1.2 ตาราง Functional Requirements ตามเกณฑ์มาตรฐาน (100% Complete)

FR-01	QtMultimedia Engine เชื่อมต่อ QMediaPlayer และ QAudioOutput ควบคุมการเล่น หยุด พัก และเร่งเสียง	PASS 100%
FR-02	Pure Vector Icons วาดไอคอน Play, Pause, Next, Previous ด้วย QPainter ใช้ไฟล์ PNG	PASS 100%
FR-03	Persistent JSON จัดเก็บ tracks, playlist, volume ลงไฟล์ storage.json เมื่อมี action ทันที	PASS 100%
FR-04	Interactive Scrubber เลื่อนตำแหน่งเพลง (Seek bar) แบบเรียลไทม์ พร้อมแสดงเวลาผ่านไปและเวลาที่เหลือ	PASS 100%
FR-05	Playlist Auto-Advance เมื่อเล่นเพลงปัจจุบันจบ ระบบข้ามไปยังเพลงถัดไปในคิวโดยอัตโนมัติ	PASS 100%

1.3 ดีไซน์ตามแนวคิด Apple Minimalism & Tech Aesthetic

การออกแบบ UI ยึดหลัก Space Black และ Frosted Titanium จัดวางเลย์เอาต์ 3 ส่วน: Sidebar นำทาง, Main View แสดงอัลบั้มและคิวเพลง, และ Under Bar ควบคุมเสียงอย่างมีเอกภาพ

บทที่ 2: สถาปัตยกรรมระบบ 3 เลเยอร์ (3-Tier Layered Architecture)

ระบบถูกออกแบบตามหลักการแยกหน้าที่ (Separation of Concerns) แบ่งออกเป็น 3 เลเยอร์หลัก เพื่อความยืดหยุ่นและการทดสอบโค้ดอย่างเป็นระบบตามมาตรฐานวิศวกรรมซอฟต์แวร์สากล

LAYER 1: PRESENTATION & UI LAYER (ui/mainWindow.py & icons.py)

- MainWindow (QMainWindow): จัดการเลย์เอาต์หน้าจอหลัก Responsive Dock Layout
- VectorIconProvider (icons.py): QPainter วาดสัญลักษณ์เวกเตอร์สดแบบ Real-time
- MediaControlsWidget: แผงควบคุม Seek bar, Volume slider, ปุ่ม Play/Pause/Skip
- PlaylistWidget: รายการคิวเพลง แสดง Track number, Title, Artist, Duration และปุ่ม Like

LAYER 2: CORE BUSINESS LOGIC & AUDIO ENGINE (core/player.py)

- AudioEngine (QMediaPlayer): ควบคุมการเล่นไฟล์เสียง .mp3, .wav, .ogg, .flac
- AudioOutputController (QAudioOutput): จัดการระดับเสียง Linear to Decibel mapping
- PlaylistManager: จัดการ Queue, Next track, Previous track, Shuffle, และ Repeat mode
- PlaybackSignals (QObject): ส่ง Signals อัปเดตสถานะ เช่น positionChanged, durationChanged

LAYER 3: DATA PERSISTENCE & STORAGE (services/storage.py)

- StorageService: จัดการไฟล์ storage.json บันทึกและโหลดข้อมูลแบบ Atomic Safe Write
- Schema Validation: ตรวจสอบความถูกต้องของข้อมูล ป้องกัน JSON เสียหาย (Corrupted)
- Backup Routine: สำรองข้อมูล storage.bak อัตโนมัติทุกครั้งก่อนเขียนไฟล์ใหม่
- OS Fallback: ตรวจสอบสิทธิ์การเขียนไฟล์ หากเกิดข้อผิดพลาดจะแจ้งเตือนผู้ใช้งานที่

บทที่ 3: โค้ดเชิงลึก Pure Vector Painting และการจัดการข้อผิดพลาด

ชอร์สโค้ดจริงที่ ใช้ในการวาดไอคอน Play/Pause ด้วย QPainter โดยไม่ใช้รูปภาพภายนอก ทำให้แอปพลิเคชันทำงานได้ 100% แม้ไม่มีไฟล์ Asset เพิ่มเติม

```
# core/icons.py - Pure Vector QPainter Engine

from PyQt6.QtGui import QPainter, QPolygonF, QColor, QPen
from PyQt6.QtCore import QPointF, Qt

def draw_play_icon(painter: QPainter, x: int, y: int, size: int, color: QColor):
    painter.setRenderHint(QPainter.RenderHint.Antialiasing, True)
    painter.setBrush(color)
    painter.setPen(Qt.PenStyle.NoPen)
    # สร้างพิกัดสามเหลี่ยม Play แบบคำนวณสมัดเรขาคณิตเวกเตอร์
    points = [
        QPointF(x + size * 0.2, y + size * 0.15),
        QPointF(x + size * 0.2, y + size * 0.85),
        QPointF(x + size * 0.85, y + size * 0.5)
    ]
    painter.drawPolygon(QPolygonF(points))
```

กลไกการเขียนโปรแกรมเชิงป้องกัน (Defensive Programming Practices)

- 1. Media File Verification: ตรวจสอบความถูกต้องของพาร์ไฟล์เสียงก่อนส่งให้ QMediaPlayer
- 2. Safe Volume Conversion: ป้องกันระดับเสียงเกินพิกัด (Clamp ค่า 0-100 ป้องกันเสียงแตก)
- 3. Index Out of Range Guard: ดักจับขอบเขตของคิวเพลง ป้องกันโปรแกรมปิดตัวเองเมื่อสิ้นสุดเพลย์ลิสต์
- 4. JSON Parse Recovery: หาก storage.json เสียหาย ระบบจะกู้คืนจาก storage.bak โดยอัตโนมัติ
- 5. Memory Leak Prevention: ทำความสะอาด QMediaPlayer instance เมื่อสลับเพลง

บทที่ 4: การคงอยู่ของข้อมูลและความปลอดภัยของสถาปัตยกรรม (Persistence & Security)

การออกแบบระบบจัดเก็บข้อมูลบนเดสก์ท็อปต้องคำนึงถึงความปลอดภัยเมื่อเครื่องคอมพิวเตอร์ดับหรือโปรแกรมถูกปิดกะทันหัน โครงสร้าง storage.json จึงถูกออกแบบให้บันทึกแบบ Atomic Write เสมอ

```
# storage.json - โครงสร้างไฟล์จัดเก็บสถานะระบบ
{
  "version": "2.5.0",
  "last_updated": "2026-09-16T12:00:00Z",
  "settings": {
    "volume": 75,
    "repeat_mode": "all",
    "shuffle": false
  },
  "playlist": [
    { "id": "t1", "title": "Celestial Symphony", "artist": "Stitch Audio", "duration": "3:45" },
    { "id": "t2", "title": "Deep Focus Beat", "artist": "Obsidian Lab", "duration": "4:12" }
  ],
  "liked_song_ids": ["t1"]
}
```

หลักการออกแบบความปลอดภัยของไฟล์ (File Security Principles)

- Atomic File Replacement: เขียนลง temp file ก่อนแทนที่ไฟล์จริง ป้องกันข้อมูลแห้ว
- Path Traversal Prevention: ตรวจสอบและ sanitize path ของไฟล์เพลง ป้องกันการเข้าถึงไฟล์ระบบ
- UTF-8 Encoding Guarantee: บันทึกด้วย UTF-8 ป้องกันชื่อเพลงภาษาไทยเกิดข้อผิดพลาดภาษาต่างดาว
- Graceful Degradе: หากไม่สามารถเขียน storage ได้ ระบบจะทำงานในโหมด In-Memory ต่อเนื่อง

บทที่ 5: การทดสอบระบบและการคอมไพล์เป็นไฟล์ Windows .exe

บันทึกผลการทดสอบการทำงานของโปรแกรม FastPlayer PRO ผ่านเกณฑ์การประเมิน 100% พร้อมคำสั่ง PyInstaller สำหรับ Build ออกเป็นโปรแกรม Standalone บน Windows

5.1 ตารางบันทึกผลการทดสอบระบบ (Test Verification Matrix)

TC-01: เปิดไฟล์เพลง .mp3 และแสดงชื่อเพลงในหน้าจอ

ผลการทดสอบ: PASSED (เสียงดังชัดเจน ไม่หน่วง)

TC-02: ทดสอบการ Seek Bar ไปข้างหน้าและถอยหลัง

ผลการทดสอบ: PASSED (ตำแหน่งเสียงตรงตามแถบเลื่อน)

TC-03: ทดสอบการปิดโปรแกรมและเปิดใหม่เพื่อกู้คืน Playlist

ผลการทดสอบ: PASSED (ข้อมูลคงอยู่ครบถ้วน 100%)

TC-04: ทดสอบปรับขนาดหน้าต่างและการเรนเดอร์ไอคอน

ผลการทดสอบ: PASSED (ไอคอนเวกเตอร์คมชัดทุกขนาด)



คำสั่ง PyInstaller Build บน Windows

```
pyinstaller --noconsole --onefile --windowed --name=FastPlayerPRO main.py
```

บันทึกการตรวจรับรองผลงาน (ENGINEERING & ACADEMIC ATTESTATION)

ผู้จัดทำ: นายเกียรติภูมิ หารศรีนาถ

รหัสนักศึกษา: 64332110242-2

สถานะ: ผ่านการทดสอบระดับโปรดิคชัน 100% พร้อมส่งมอบในรูปแบบ E-book และซอร์สโค้ด

รับรองความถูกต้องของสถาปัตยกรรมระบบและซอร์สโค้ดตามเกณฑ์การประเมินวิศวกรรมซอฟต์แวร์